

10_research팀_김서영_요약본

논문 1. Efficient Memory Management for LLM Serving with PagedAttention | Kwon et al., 2023

배경 및 문제

- **목표:** KV cache 메모리를 효율적으로 관리해 LLM serving throughput 향상 (A100 40GB 기준 KV cache > 30% 차지)
- **LLM 동작:** Autoregressive Transformer → 매 step마다 이전 token의 KV cache 필요 → memory-bound, GPU 활용 낮음
- **기존 한계:** FasterTransformer·Orca는 KV cache를 연속 메모리(contiguous)에 저장 → 두 가지 비효율 발생
 - Internal fragmentation: 최대 출력 길이 기준으로 pre-allocation → 실제 출력이 짧으면 낭비
 - External fragmentation: 요청마다 크기가 달라 메모리 할당자 단편화 / Memory sharing 불가
- **수치:** 기존 시스템의 실제 KV cache 활용률은 20.4% ~ 38.2%에 불과

방법: PagedAttention + vLLM

- **핵심 아이디어:** OS virtual memory & paging에서 영감 → blocks=pages, tokens=bytes, requests=processes
- 각 sequence의 KV cache를 고정 크기의 KV block으로 분할 → non-contiguous physical memory에 저장
- Block table로 logical ↔ physical block 매핑 / 필요할 때만 동적 할당 → 메모리 낭비 마지막 block 1개 이내
- **효과:** internal fragmentation 완화 / external fragmentation 제거 / block 단위 memory sharing 가능

디코딩 사례

- **Parallel Sampling:** 동일 prompt의 KV cache를 물리 블록에 저장 후 공유 (copy-on-write), long prompt일수록 절약 효과 큼
- **Beam Search:** 중간 block까지 candidate 간 공유 가능 / sharing pattern이 동적으로 변화 (OS fork tree 유사)
- **Shared Prefix:** 동일 system prompt를 가진 요청들이 미리 캐싱된 블록을 직접 매핑 → prefix 계산 중복 제거

Scheduling & Preemption & Distributed Execution

- FCFS 정책 / GPU 블록 부족 시 all-or-nothing eviction / 복구: Swapping 또는 Recomputation

- 대형 모델: Megatron-LM 방식 tensor model parallelism (SPMD) / 단일 KV cache manager 로 centralized 관리

실험 결과

- OPT-13B/66B/175B, LLaMA-13B / ShareGPT-Alpaca 데이터셋 / FasterTransformer·Orca 대비 비교
- **Basic Sampling:** vLLM 이 Orca 대비 ShareGPT 4.3x, Alpaca 18.9x 더 많은 batched requests 처리
- **전체 결과:** FasterTransformer·Orca 대비 2~4x throughput 향상 (모델 정확도 변화 없음)
- **Block size:** 16 이 최적 (GPU 효율 + fragmentation 최소화 균형) / Attention kernel overhead 20~26%이나 end-to-end 성능 압도적 우위

논문 2. Adam: A Method for Stochastic Optimization | Kingma & Ba, 2014

배경 및 문제

- **목적:** 딥러닝 loss function 은 본질적으로 stochastic → 매 미니배치마다 그래디언트가 달라짐, dropout 등 추가 noise 주입
- **필요:** 1 차 미분 기반 방법은 빠르지만 방향 정보 제한적 / 2 차 미분(Hessian)은 $O(d^2 \sim d^3)$ 로 비현실적
- **결론:** 효율적이고 noise 에 강건한 1 차 stochastic 옵티마이저 필요

선행 연구

- **AdaGrad (2011):** 누적 제곱 그래디언트로 학습률 자동 조절 → sparse gradient 에 강함 / 한계: h 가 단조 증가해 장기 학습 시 학습률 0 수렴
- **RMSProp (2012):** 지수 이동평균(EMA)으로 AdaGrad 의 학습률 소실 문제 해결 ($\rho \approx 0.9 \rightarrow$ 직전 ~10 스텝 분산 추적)
- **Adam:** Momentum(방향)과 RMSProp(스텝 크기)을 통합 → Adaptive Moment Estimation

알고리즘

- **파라미터:** $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$ | 초기화: $m_0 = 0, v_0 = 0$
- ① $g_t = \nabla \theta f_t(\theta_{t-1})$ — 미니배치 그래디언트 계산
- ② $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ [1 차 EMA] / $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ [2 차 EMA]
- ③ $\hat{m}_t = m_t / (1 - \beta_1^t), \hat{v}_t = v_t / (1 - \beta_2^t)$ — 편향 보정
- ④ $\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$ — 파라미터 업데이트

핵심 이론

- **편향 보정의 필요성:** $m_0 = 0, v_0 = 0$ 초기화 시 초기 추정치가 0 으로 편향 / $t = 1, \beta_1 = 0.9$ 에서 $1 - \beta_1^t = 0.1 \rightarrow$ 보정 없으면 실제값의 10%만 반영
- **SNR 해석:** $\hat{m}_t / \sqrt{\hat{v}_t} \approx$ 신호 대 잡음비(SNR) $\rightarrow$ SNR $\uparrow$ 큰 스텝 / SNR $\downarrow$ 작은 스텝 — 자동 신뢰도 조절
- **Step size 상한:** $|\Delta\theta_t| \leq \alpha$ — 학습률이 무한대로 발산하지 않음 / 최적값 근처에서 자연스럽게 annealing
- **효율적 구현:** $\alpha_t = \alpha \cdot \sqrt{(1 - \beta_2^t) / (1 - \beta_1^t)}$ 로 편향 보정을 학습률에 흡수 $\rightarrow \hat{m}, \hat{v}$ 별도 저장 불필요, 나눗셈 1 회로 단축 (PyTorch/TF 채택)

실험 결과

- **MNIST (Logistic Regression):** Adam 이 AdaGrad 보다 빠르게 수렴 / SGD 와 유사한 속도 $\rightarrow$ dense feature 환경에서도 안정적
- **IMDB BoW (sparse, 10,000 차원 + dropout):** Adam, RMSProp, AdaGrad 모두 우수 / SGD 보다 빠른 수렴
- **MLP (MNIST, 2layer $\times$ 1000):** Adam 이 가장 낮은 cost / SFO(2 차 근사)보다 iterations-wall-time 모두 우위
- **CNN (CIFAR-10, 단기 3 epoch):** Adam 이 가장 빠른 cost 감소 / 장기 45 epoch: AdaGrad 는 학습률 소실로 정체, Adam 단기·장기 모두 최저 cost 달성
- **Bias Correction 검증:** $\beta_2=0.9999$ 에서 미적용 시 초기 발산 / 10 epoch 에서 차이 결정적 / bias correction 은 초반 안정성에 필수

Ablation Studies

- **AdaMax:** v_t 를 L_∞ -norm (max)으로 대체 $\rightarrow u_t = \max(\beta_2 \cdot u_{t-1}, |g_t|) / \epsilon$ 불필요, v_t bias correction 불필요 / 권장: $\alpha = 0.002, \beta_1 = 0.9, \beta_2 = 0.999$
- **Temporal Averaging:** $\theta_t = \beta_2 \cdot \theta_{t-1} + (1 - \beta_2) \cdot \theta_t \rightarrow$ 메모리 1 배 추가로 수렴 안정성·일반화 성능 향상 / Adam·AdaMax 에 그대로 결합 가능

결론

- 3 대 강점: ① $O(d)$ 효율성 ② 그래디언트 대각 재스케일링에 견고 ③ 기본값으로 대부분 작동
- 편향 보정 = 작지만 결정적 기여 — Momentum + RMSProp 장점을 완전 통합
- BERT·GPT·ResNet·GAN 등의 사실상 표준 옵티마이저 / 120,000+ Citations